

Killing Flaky Tests & Running AI Agents for Hours

A real-world engineering pattern for building test automation that is **reliable**, **autonomous**, and **accurate** — powered by a large-context AI model and a disciplined agent **harness**.



Reliable

Flakiness engineered out



Durable

Agents run for hours



Accurate

Results you can trust

A vendor-neutral reference pattern — applicable to any web application and any test suite.

Two failure modes that break automation



Flaky Tests

Tests that pass and fail without any code change destroy trust in the whole suite. A classic root cause is **authentication state**: when a token-based auth provider stores sessions in browser storage and rotates refresh tokens, reusing a saved session goes stale and fails intermittently — and logging in concurrently trips rate limits.



Long-Running Agents

An AI agent that generates and repairs many tests can run for a long time. Without help it **loses earlier context**, repeats work, drifts off task, or dies mid-run — wasting compute and producing inconsistent output.

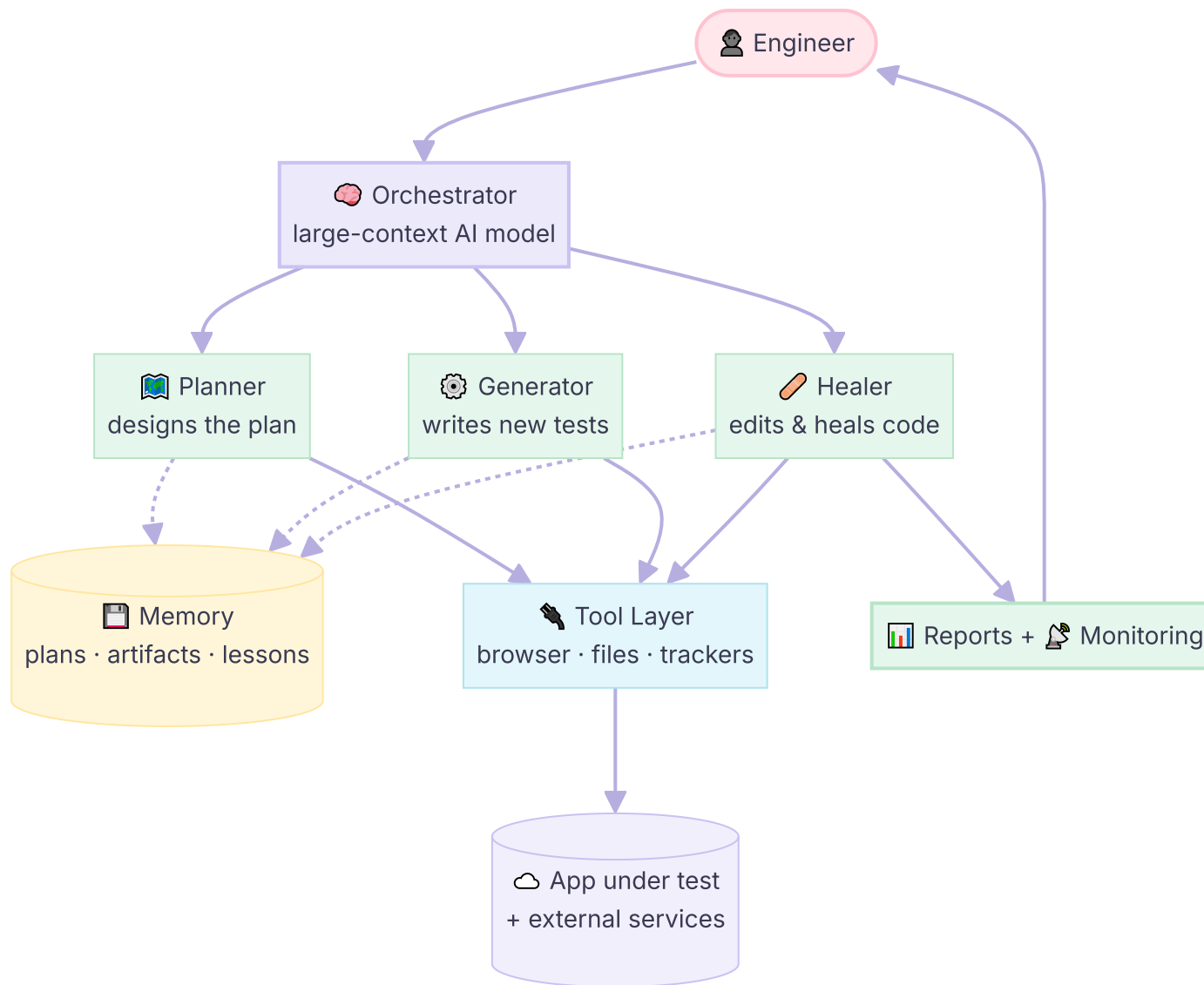


The goal

Make every run **deterministic** (no flakiness), **durable** (survives long autonomous sessions), and **accurate** (a quality signal you can act on).

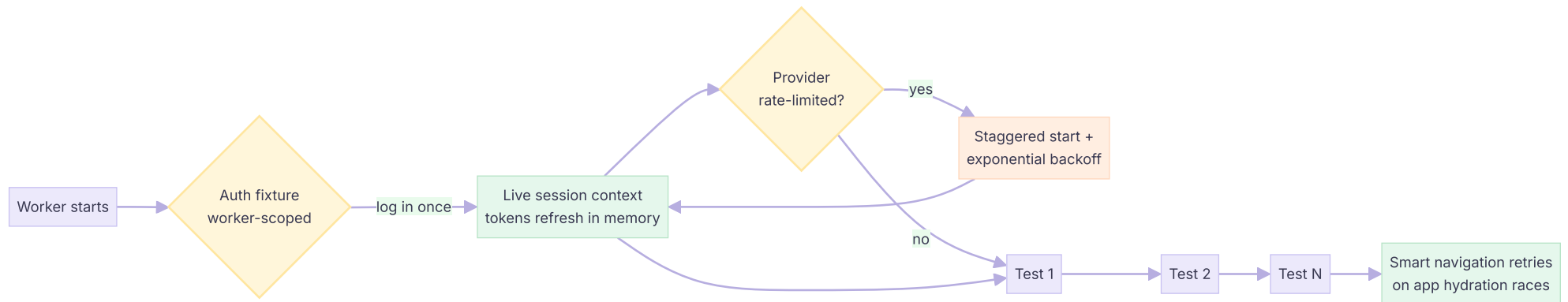
One orchestrator, three specialist agents

A large-context AI orchestrator decomposes a requirement and delegates to least-privilege sub-agents that collaborate through a shared file "blackboard".



Least privilege: the Planner & Generator hold no code-editing tools, so they can author plans and new tests but never corrupt existing code — only the Healer edits files.

How to eliminate flaky tests



Log in once per worker

Each parallel worker authenticates **once** into a live session kept alive for its lifetime — tokens refresh in memory, never from a stale file.



Backoff & stagger

Staggered startup + exponential backoff absorbs the auth provider's concurrent-login rate limits.



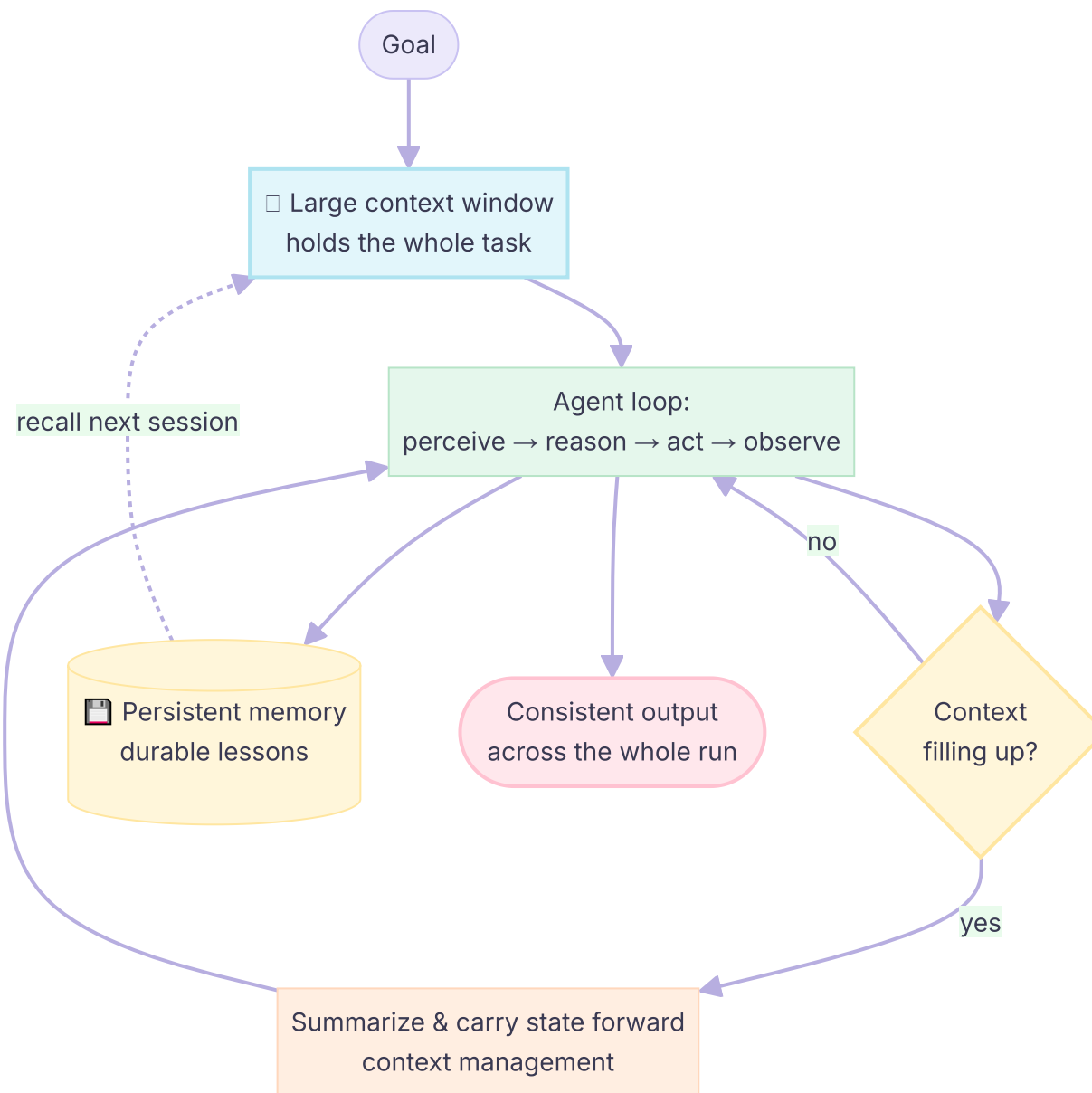
No blind waits

Tie every wait to a **real signal** (an element, a URL, a response) — never a fixed sleep that masks timing bugs.

*Principle: a suite should pass at **zero retries**. Keep one retry only as a safety net for genuine infrastructure blips — never as a flake mask.*

How long-running agents stay on track

The **model** and the **harness** work together so an agent can run for hours without losing the thread.



Large context window

A model with a very large context (e.g. a 1M-token window) keeps the whole task —

Context management

When context grows, the harness summarizes and carries state forward so

Persistent memory

Durable lessons are written to memory and recalled in future runs, so the agent gets

plan, code and failures — in view at once.

work continues seamlessly past any single window.

smarter over time.

Going further: a dedicated long-running harness adds heartbeats, checkpoints & auto-recovery for multi-hour, unattended runs.

Why the results become trustworthy

✗ Without the pattern

- Random pass/fail; retries hide real bugs
- Selectors patched inside tests → drift & rot
- Agents forget context and repeat work
- Numbers nobody trusts

✓ With the pattern

- Deterministic auth & real-signal waits
- Fixes layered — locators live in Page Objects
- Self-healing repairs the right layer
- Evidence-backed, auditable results



Layered fixes

Repair at the correct abstraction (Page Object, fixture, config) — never patch a selector inside a test.



Evidence first

Capture screenshots, traces, console & network for every failure so a result is provable, not asserted.



Full traceability

Tie each finding back to a requirement and forward to a tracked ticket — quality you can audit end to end.

Agentic QA is practical today — not someday

By pairing a **large-context AI model** with a disciplined agent **harness**, flakiness is engineered out, long autonomous runs stay coherent, and the quality signal becomes accurate enough to act on with confidence.



Reliable

Single-login sessions, no blind waits, zero-retry green.



Durable

Large context + context management + persistent memory.



Accurate

Layered fixes, evidence-first, full traceability.

► A vendor-neutral reference pattern you can apply to any web app and any test suite.

